

Functional Specification Document (FSD)

Phase / Category	Project Documentation	Date	1/10/2026
Team ID	6ab9466dc23107aaaa73904b	Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	10 Marks		

Functional Specification Document (FSD)

Project Title: Implement Client Script & UI Policy (Incident)

Team ID: 6ab9466dc23107aaaa73904b | **Date:** 1/10/2026

1. Introduction

Project Title: Implement Client Script & UI Policy (Incident)

Author / Team ID: 6ab9466dc23107aaaa73904b

Platform: ServiceNow Washington DC / Xanadu Enterprise ITSM

Document Version: 1.0 (Final Approved Release)

2. Project Overview

Purpose: Enforce data integrity, automated field values, and conditional mandatory rules on the ServiceNow Incident table at the client interface level.

Key Features:

- Declarative UI Policy enforcing Assignment Group mandatory when Impact is High.
- UI Policy Action making Urgency read-only to avoid accidental downgrades.
- Client Script (onChange) dynamically calculating and populating Urgency = 1.
- Client Script (onSubmit) validating that Assigned To is not blank before database commit.
- Client Script (onCellEdit) securing list view against direct State field edits.

3. Architecture

The solution utilizes the ServiceNow GlideForm client API operating inside the browser DOM. The declarative UI Policy engine evaluates conditions upon field changes and form load. Procedural Client Scripts handle event-driven business logic and validation hooks.

4. Setup & Configuration Instructions

1. Log in to ServiceNow instance as Administrator or ITIL Admin.
2. Navigate to System UI → UI Policies and click **New**.
3. Set Name: *High Impact Control*, Table: *Incident*, Condition: *Impact is 1 - High*, Reverse if false: *true*. Submit.
4. In UI Policy Actions related list, create Action 1: Field `assignment_group`, Mandatory: *true*.
5. Create Action 2: Field `urgency`, Read-only: *true*, Visible: *Leave unchanged*.

6. Navigate to System UI → Client Scripts and click **New**.
7. Create onChange Script: Name: *Auto set urgency for high impact*, Table: *Incident*, Type: *onChange*, Field: *Impact*. Paste script and save.
8. Create onSubmit Script: Name: *Prevent save if Assigned To missing*, Table: *Incident*, Type: *onSubmit*. Paste script and save.
9. Create onCellEdit Script: Name: *Prevent state change via list edit*, Table: *Incident*, Type: *onCellEdit*, Field: *State*. Paste script and save.

5. Configuration Artefact Hierarchy

Artifact Name	Type	Scope / Table	Trigger Condition / Event	Action Taken
High Impact Control	UI Policy	incident	impact == 1	Controls mandatory and read-only states dynamically.
assignment_group Action	UI Policy Action	incident	Policy condition met	Mandatory = true
urgency Action	UI Policy Action	incident	Policy condition met	Read-only = true
Auto set urgency for high impact	Client Script	incident	onChange(impact)	Sets Urgency to '1' and displays info message.
Prevent save if Assigned To missing	Client Script	incident	onSubmit()	Blocks save if Assigned To is empty on High impact.
Prevent state change via list edit	Client Script	incident	onCellEdit(state)	Alerts user and returns callback(false).

6. Source Code Documentation

6.1 onChange Script: Auto set urgency for high impact

```
function onChange(control, oldValue, newValue, isLoading) {  
    // Exit if form is loading or value is empty  
    if (isLoading || newValue == '') {  
        return;  
    }  
    // Check if impact is set to 1 (High)  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

6.2 onSubmit Script: Prevent save if Assigned To missing

```
function onSubmit() {  
    // Verify high impact and missing assignee  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == '') {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false; // Prevent form submission  
    }  
    return true; // Allow form submission  
}
```

6.3 onCellEdit Script: Prevent state change via list edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    // Notify user that state cannot be updated from list  
    alert('State cannot be updated using list editing. Please open the Incident.');    callback(false); // Disallow cell update  
}
```

7. Visual Evidence & Configuration Screenshots

The screenshot shows the 'UI Policy - High Impact Control' configuration page in ServiceNow. The left sidebar contains a navigation menu with categories like 'System Mobile', 'Mobile UI', 'Service Catalog', 'Catalog Administration', 'System Diagnostics', 'Session Debug', 'System UI', 'UI Policies', 'MID Server', 'MID Security Policy', 'System Web Services', and 'SOAP Security Policies'. The main content area is titled 'UI Policy - High Impact Control View: Advanced'. It includes a 'Table' dropdown set to 'Incident [incident]', an 'Application' dropdown set to 'Global', and an 'Active' checkbox checked. The 'Short description' field is set to 'High Impact Control', and the 'Order' is 100. Below these fields, there are two tabs: 'When to Apply' and 'Script'. The 'When to Apply' tab is active, showing a 'Conditions' section with a single condition: 'Impact' is '1 - High'. There are also checkboxes for 'Global' (checked), 'Reverse if false' (checked), 'On load' (checked), and 'Inherit' (unchecked). A note at the bottom states: 'If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific'.

Figure 6.1: High Impact Control UI Policy Setup

The screenshot shows the 'Client Script - Prevent save if assigned to missing' configuration page in ServiceNow. The left sidebar contains a navigation menu with categories like 'Service Catalog', 'Catalog Administration', 'Catalog Client Scripts', 'System Definition', 'Client Scripts', 'System Logs', and 'Client Transactions'. The main content area is titled 'Client Script - Prevent save if assigned to missing'. It includes a 'Name' field set to 'Prevent save if assigned to missing', a 'Table' dropdown set to 'Incident [incident]', a 'UI Type' dropdown set to 'Desktop', and a 'Type' dropdown set to 'onSubmit'. The 'Application' dropdown is set to 'Global', and the 'Active' checkbox is checked. Below these fields, there are fields for 'Description' and 'Messages'. The 'Script' field contains the following code:

```

1 function onSubmit() {
2   if (g_form.getValue('impact') == '1' &&
3       g_form.getValue('assigned_to') == '') {
4     g_form.showErrorMessage(
5       'assigned_to',
6       'Assigned To is mandatory for High impact incidents.'
7     );
8     return false;
9   }
10  return true;
11 }
12
13
14

```

The 'Isolate script' checkbox is checked. At the bottom, there are 'Update' and 'Delete' buttons.

Figure 6.2: onSubmit Validation Script Configuration

servicenow All

client script

FAVORITES No Results

ALL RESULTS

- Service Catalog
- Catalog Administration
- Catalog Client Scripts
- System Definition
- Client Scripts

Client Script - Prevent state change via list edit

Search

Update Delete

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: Prevent state change via list edit

Table: Incident [Incident]

Application: Global

UI Type: Desktop

Type: onCellEdit

Field name: State

Active: ☒

Inherited: ☐

Global: ☒

Description:

Messages:

Script

```

1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
2
3     alert('State cannot be changed from the list. Please open the Incident.');
```

```

4
5     // Cancel the list edit
6     callback(false, oldValues);
7 }
8
9
```

Isolate script: ☒

Figure 6.3: onCellEdit List Control Script Configuration

8. Known Issues & Maintenance Guidelines

- **Maintenance Guideline:** Whenever adding new incident states, ensure onCellEdit scripts are reviewed to prevent locking administrative overrides.
- **Best Practice:** Always maintain `Reverse if false = true` on UI Policies to prevent orphaned field locks when agents toggle between priorities.